

Vorlesung (WS 2016/17) *Softwarekonstruktion*

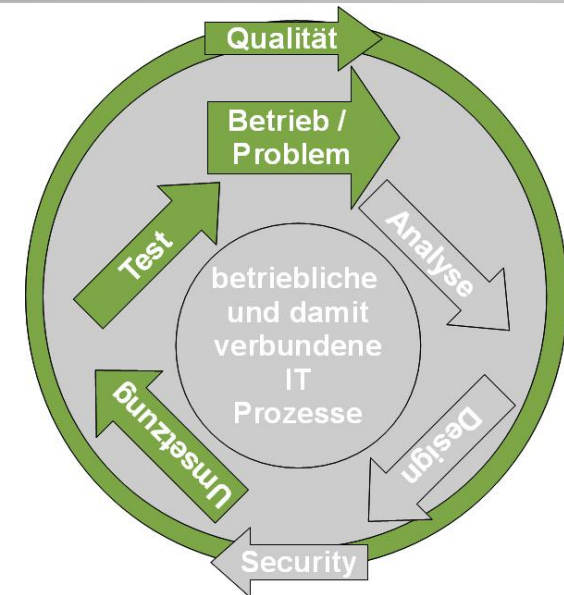
Dr. Boris Düdder

Lehrstuhl XIV – Software Engineering
TU Dortmund, Fakultät Informatik

Teil 2.2: Softwariemetriken und Kostenschätzung

v. 05.12.2016

- Modellgetriebene SW-Entwicklung
- Qualitätsmanagement
- **Testen**
 - Grundlagen Softwareverifikation
 - **Softwaremetriken**
 - Black-Box-Test
 - White-Box-Test
 - Testen im Softwarelebenszyklus



[inkl. Beiträge von Prof. Martin Glinz, Universität Zürich und Prof. Ian Sommerville, Univ. St. Andrews]

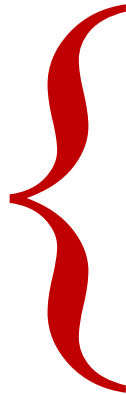
Literatur (s. Vorlesungswebseite):

- Andreas Spillner, Tilo Linz: Basiswissen Softwaretest.
- Eike Riedemann: Testmethoden für sequentielle und nebenläufige Software-Systeme.

- Verständnis entwickeln für Softwaremetriken und Metriken zum Schätzen.
- Geeignete Metriken klassifizieren, vergleichen und auswählen können.
- Metriken auf Software anwenden und Resultate interpretieren können.

- **Vorheriger Abschnitt:** Einführung Softwareverifikation
- **Dieser Abschnitt:** Bestimmung der Komplexität des Programmes
 - Softwariemetriken: Zyklomatische Zahl
 - Werkzeugunterstützung
 - Aufwandsschätzung

2.2 Software- metriken



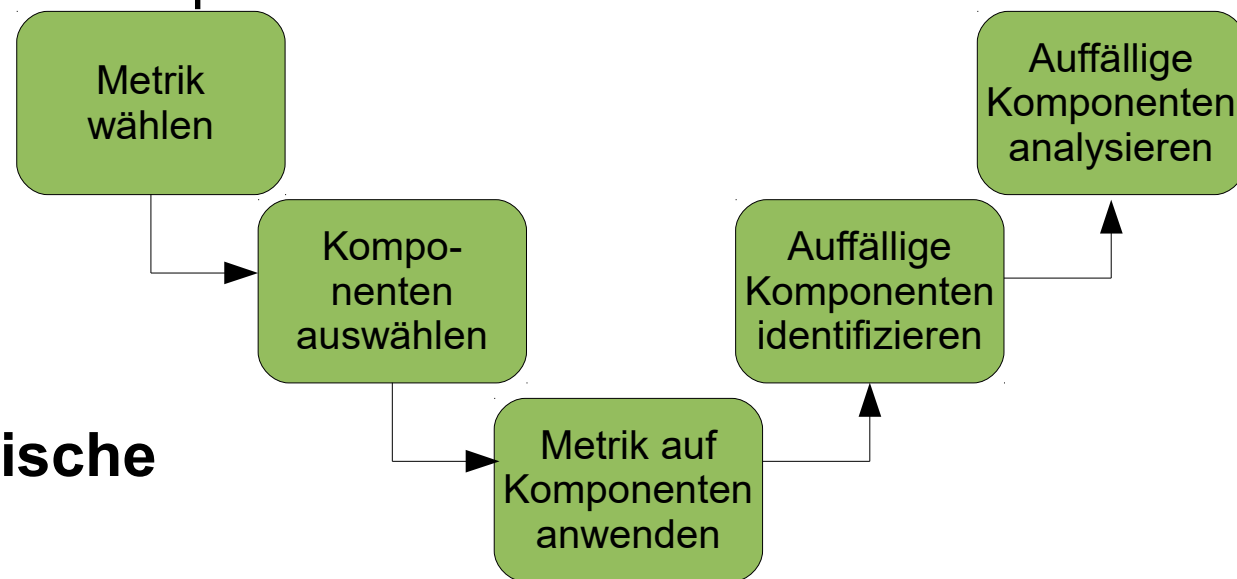
Motivation und Überblick

Zyklomatische Komplexität

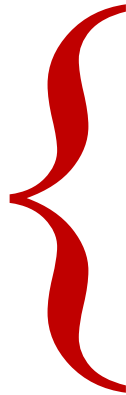
Objektorientierte Metriken-Suite

Werkzeugunterstützung

- Vollständiges Austesten i.A. unmöglich => **Prioritäten** setzen.
- Schwerpunkt auf besonders **fehleranfällige** Teile der Software legen !
- Wie diese Teile effizient identifizieren ?
- Fehleranfälligkeit korreliert mit **Komplexität** von Softwarekomponente.
- Komplexität der Softwarekomponenten mit **Metriken** ermitteln.
- Metrikwerte für verschiedene Komponenten und mit historischen Daten **vergleichen**.
- Anomale Werte können auf Qualitätsprobleme hinweisen
=> intensiver testen.
- Hier ein Beispiel: **zyklomatische Komplexität**.



2.2 Software- metriken



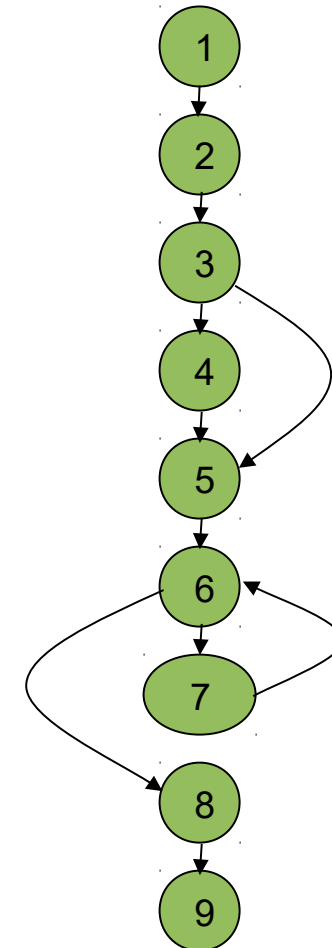
Motivation und Überblick

Zyklomatische Komplexität

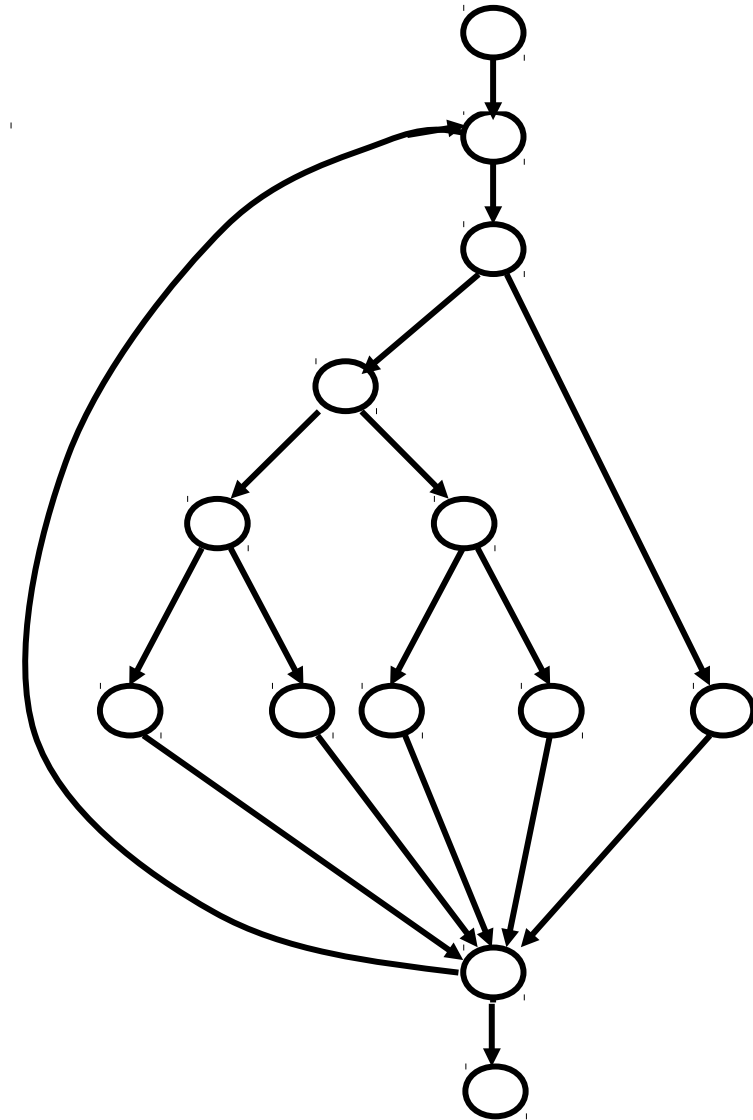
Objektorientierte Metriken-Suite

Werkzeugunterstützung

- **Zyklomatische Komplexität**
(„McCabe-Metrik“):
Misst Komplexität eines Programmes auf Basis
des Kontrollflussgraphen.
- **Zyklomatische Komplexität** des
Kontrollflussgraphen G : $v(G) = e - n + 2$
für Kantenanzahl e und Knotenanzahl n
- **Beispiel:** $v(G) = ?$



- **Bemerkung:** Berechnung von $v(G)$ in Programmiersprachen mit geschlossenen Ablaufkonstrukten:
 - Zähle alle Verzweigungen und Schleifen (**if**, **while**, **for**, etc.).
 - Addiere für jede Auswahl-Anweisung (**switch**, **CASE**) die Zahl der Fälle - 1.
 - Addiere 1.
- **Hier:** ? + 1
- **Entspricht McCabe-Formel:**
 $v = ?$, $n = ?$
 $v(G) = e - n + 2 = ?$



Bislang Annahme: Programm hat nur ein Endpunkt.

Zyklomatische Komplexität des Kontrollflussgraphen G eines Programms **mit mehreren Endpunkten:**

$$v(G) = e - n + p + 1$$

- e Zahl der Kanten
- n Zahl der Knoten
- p Zahl der Endpunkte des Programms

[NB: Annahme weiterhin: Nur ein Startpunkt.]

Zyklomatische Komplexität > 10 nach McCabe **nicht tolerabel**.

- Überarbeitung des Programnteils !
- Messwert 6 im Beispiel liegt im Bereich $\rightarrow$ nach McCabe akzeptabel
- Z.T. Messwerte bis 15 ausnahmsweise akzeptiert (dokumentierte Begründung).

Für Wartbarkeit: Verständlichkeit eines Programmstücks wichtig.

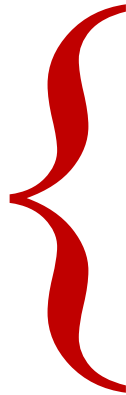
- Ermittelte **zyklomatische Komplexität hoch**:
 - $\rightarrow$ Nachvollziehen des Ablaufs des Programmstücks schwierig.
 - $\rightarrow$ **Schlechte Verständlichkeit.**

Auskunft über Testaufwand:

- Zykl. Komplexität = **Anzahl unabhängiger Pfade.**
- Zykl. Komplexität – 1 = **Anzahl Entscheidungen** im Kontrollflussgraph.
- **100%-ige Ausführung aller Anweisungen** und Verzweigungsmöglichkeiten eines Programms verlangt.
→ Einmaliger Durchlauf unabhängiger Pfade durch Kontrollflussgraphen.
- Zykl. Komplexität: **Obere Grenze für Anzahl benötigter Testfälle** zur Erreichung dieses Kriteriums.

- Problem der **Validität**:
 - Komplexität Spaghetti-Programm = Komplexität wohlstrukturiertes Programm gleichen Problems.
 - Auch intuitiv gleich komplex ?
- Eignet sich Maß als **Indikator für Fehleranfälligkeit** ?
 - Nicht besser als NCSS (Kafura und Canning, 1985).
- Skala: **Verhältnisskala**, aber **nicht additiv**:
 - Aneinanderreihung zweier Programmstücke mit Komplexitäten v_1 und v_2 : Komplexität $v_1 + v_2 - 1$
 - **Kontraintuitive Eigenschaft**.

2.2 Software- metriken



Motivation und Überblick

Zyklomatische Komplexität

Objektorientierte Metriken-Suite

Werkzeugunterstützung

Die objektorientierte Metriken-Suite „CK“ (1)

Softwarekonstruktion
WS 2016/17



LEHRSTUHL 14
SOFTWARE ENGINEERING

Softwaremetrik	Beschreibung
Gewichtete Methoden pro Klasse (Weighted Method per Class, WMC)	Anzahl definierter Methoden in jeder Klasse , gewichtet durch Komplexität: $WMC = \sum C(i)$ mit $C(i)$ = Komplexität von Methode i Komplexe Objekte: Verständnis schwer.
Tiefe des Vererbungsbaums (Depth of Inheritance Tree, DIT)	Maximale Tiefe der Generalisierungshierarchie. → Anzahl Ebenen im Vererbungsbaum. Je tiefer der Baum, desto komplexer das Design: Viele Klassen verstehen, um unterstes Blatt des Vererbungsbaums nachzuvollziehen.
Zahl der Kinder (Number of Children, NOC)	Anzahl direkter Unterklassen. • Hoher NOC-Wert = hohe Wiederverwendung → Validierung der Basisklassen aufwändig, wegen großer Zahl abhängiger Unterklassen.

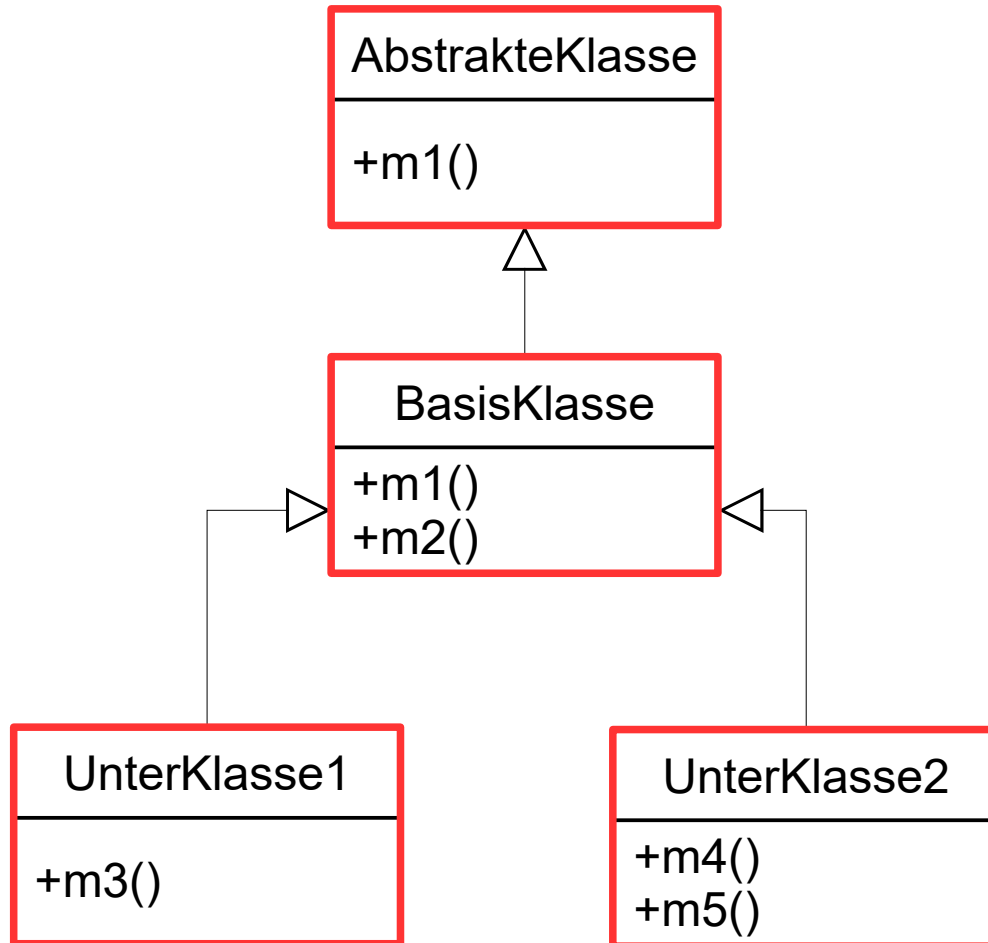
Die objektorientierte Metriken-Suite „CK“ (2)

Softwarekonstruktion
WS 2016/17



LEHRSTUHL 14
SOFTWARE ENGINEERING

Softwaremetrik	Beschreibung
Kopplung von Klassen (CBO)	Methoden von C benutzen Methoden/Variablen von D → Klassen C und D gekoppelt. CBO: Maß für Anzahl der Kopplungen . Hoher CBO-Wert: Klassen voneinander sehr abhängig → Änderung einer Klasse betrifft viele Klassen im Programm.
Reaktion einer Klasse (RFC)	Maß für Anzahl Methoden: als Antwort auf Nachricht an umgebene Klasse ausführbar. Hoher RFC-Wert: Klasse komplexer und fehleranfälliger.
Mangel an Zusammenhalt in Methoden (Lack of Cohesion of Methods, LCOM)	Anzahl gemeinsam benutzter Instanzvariablen von Methoden einer Klasse. Verschiedene Arten von Metriken → Informationen zu anderen Metriken lieferbar.

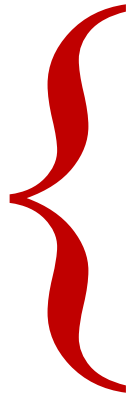


- AbstrakteKlasse
 - WMC = 1; DIT = 0;
NOC = 1
- BasisKlasse
 - WMC = 2; DIT = 1;
NOC = 2
- UnterKlasse1
 - WMC = 1; DIT = 2;
NOC = 0
- UnterKlasse2
 - WMC = 2; DIT = 2;
NOC = 0

(hier wird angenommen, dass jede Methode die Komplexität $C = 1$ hat)

Komplexitätsmaße: Indikator für Fehleranfälligkeit und Pflegbarkeit.
Welches **Problem** ergibt sich, wenn Programmierer am
Komplexitätsmaß gemessen wird, um beide Eigenschaften zu
steuern ?

2.2 Software- metriken



Motivation und Überblick

Zyklomatische Komplexität

Objektorientierte Metriken-Suite

Werkzeugunterstützung

Testwell CMTJava <http://www.testwell.fi/cmtjdesc.html>

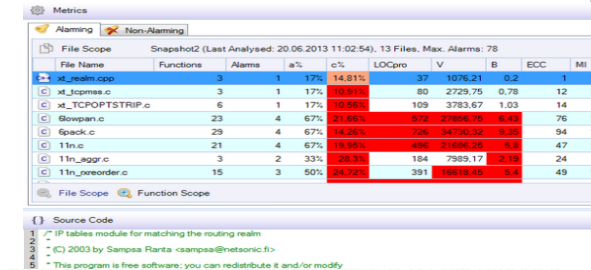
- *Unterstützte Metriken:* Zeilenmetriken, Metrik, McCabe Zyklomatische Wartungsaufwand.

Eclipse Metrics Plugin (Freeware) <http://eclipse-metrics.sourceforge.net>

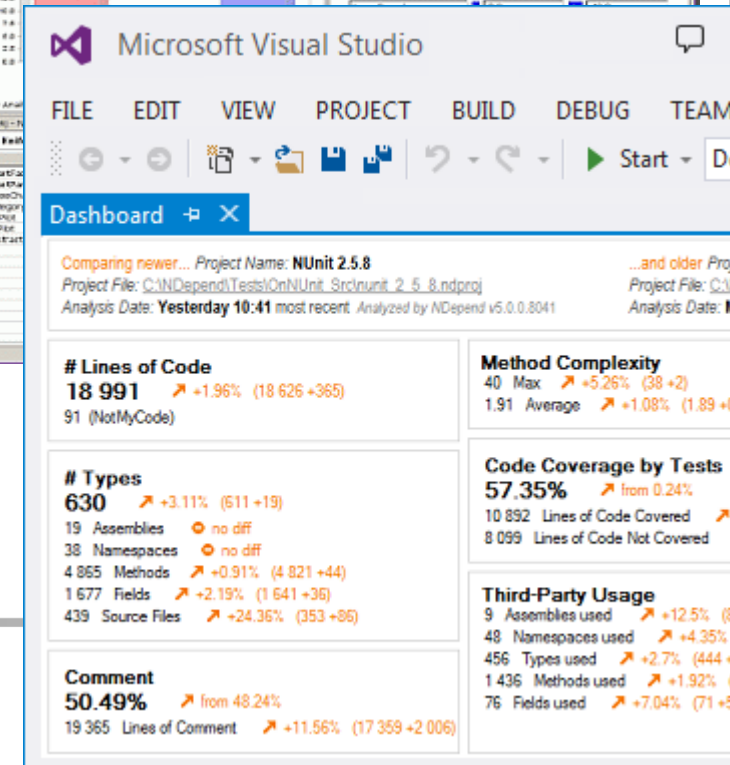
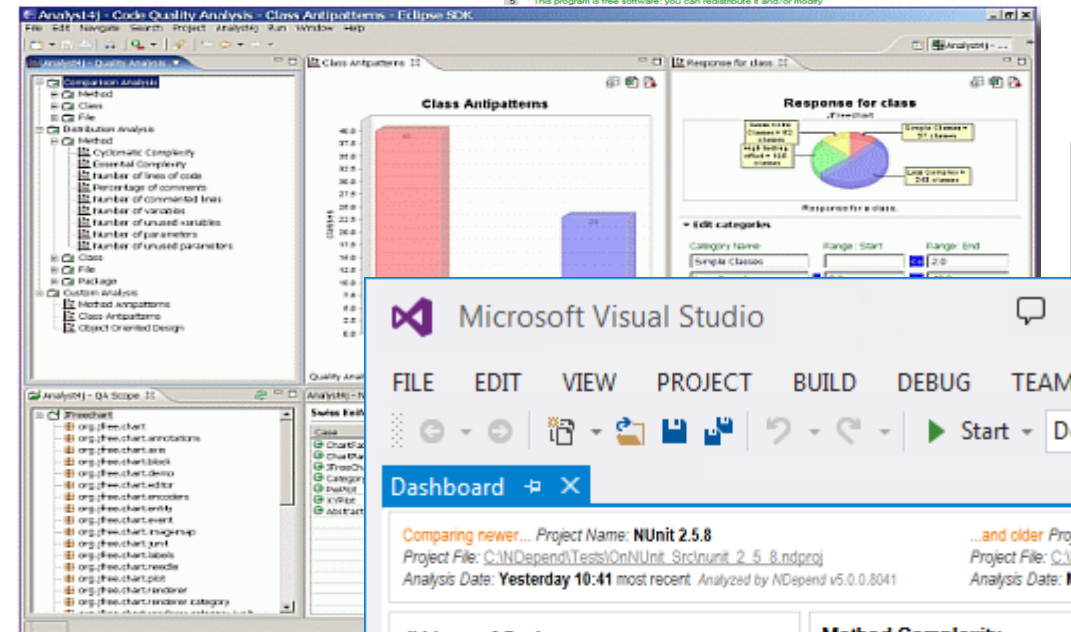
- *Unterstützte Metriken:* McCabe Zyklomatische Komplexität.

Ndepend (Testversion) <http://ndepend.com>

- *Unterstützte Metriken:*
 - Zyklomatische Komplexität
 - Schätzung des Entwicklungsaufwands.
 - Erkennung von großen Methoden und Klassen.



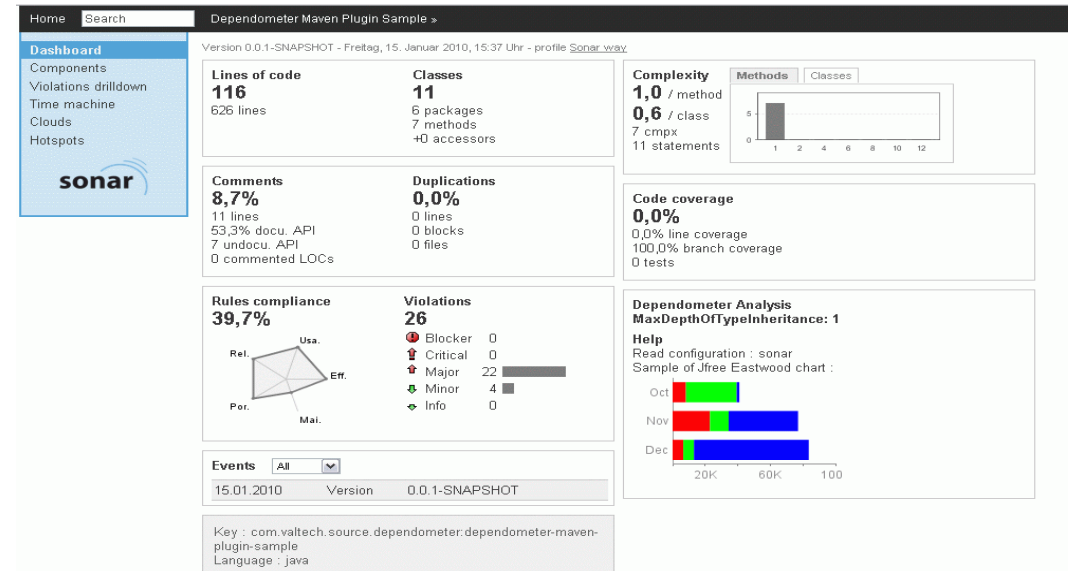
File Name	Functions	Alarms	a%	c%	LOCpro	V	B	ECC	MI
C:\jre\bin\app	3	1	17%	14.81%	37	1076.21	0.2	12	14
C:\jre\bin\c	3	1	17%	14.81%	80	2729.75	0.78	12	14
C:\jre\bin\TCTOPOSTRIP.c	6	1	17%	14.81%	109	3783.67	1.03	14	17
C:\jre\bin\Bopen.c	23	4	67%	21.86%	672	2768.25	5.43	76	10
C:\jre\bin\6pack.c	29	4	67%	14.26%	738	34730.32	8.36	94	16
C:\jre\bin\11n.c	21	4	67%	18.85%	458	2188.25	5.8	47	16
C:\jre\bin\11n_aggr.c	3	2	33%	28.3%	184	7889.17	2.18	24	5
C:\jre\bin\11n_reorder.c	15	3	50%	24.32%	351	18818.45	8.4	49	10



Sonar Source (Freeware)

<http://sonarsource.com>

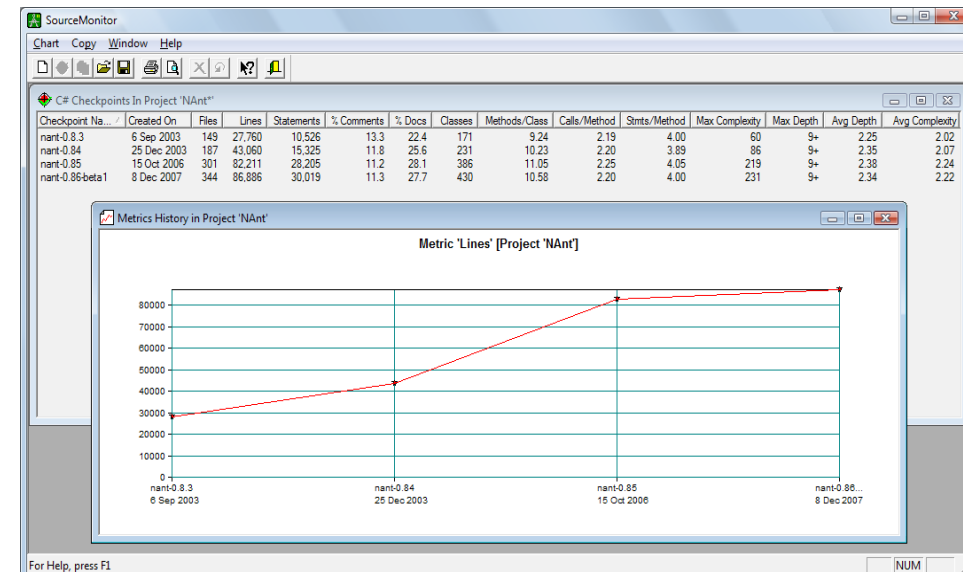
- *Unterstützte Metriken:* **McCabe Zyklomatische Komplexität.**



Source Monitor (Freeware)

<http://www.campwoodsw.com/sourcemonitor.html>

- *Unterstützte Metriken:* **Zyklomatische Komplexität.**



- Bestandteil der Planung eines Softwareprojekts
- Geschätzte Projektgrößen
 - Personen
 - Zeit
 - Ressourcen
- Am Ende: Kosten, Termine, Ressourcen (!)

- Sachkosten
 - Hardware, Lizenzen, Fortbildung, Reisekosten, ...
- Personalkosten
- kaufmännische Aufschläge
 - Realisierungsrisiko
 - Sicherheitsaufschlag für Fehleinschätzung
 - Vorfinanzierungskosten
 - Personalkostensteigerung oder Wechselkursrisiken

- Analogie
- Price-to-win („Wie teuer darf es sein, um Auftrag zu erhalten?“)
- Program Evaluation and Review Technique (PERT)
- Delphi-Methode (Schätzklausur)
- **COCOMO II**
- **Use Case Points**
- The Planning Game

COCOMO

(Constructive Cost Model)

- Entwickelt von Barry W. Boehm, 1981
- Verfahren sehr gut für schnelle Schätzung!
- Gegeben Projektgröße in 1000 Zeilencode (kLoC)
- Aufwand in Personemonaten [PM] = $m \cdot (\text{kLoC})^n$
- Projektkomplexität bedingt m, n
 - Einfach: $2,4 \cdot (\text{kLoC})^{1,05}$
 - Mittel: $3,0 \cdot (\text{kLoC})^{1,12}$
 - Schwer: $3,6 \cdot (\text{kLoC})^{1,2}$

- Personenmonate nicht einfach auf Personen aufteilbar
 - 300 PM auf 3000 Mitarbeiter → Projektende in 2,5d??
- Daher Entwicklungszeit TDEV in Monaten
 - Einfach: $TDEV [\text{Monate}] = 2,5 * \text{Aufwand}^{0,38}$
 - Mittel: $TDEV [\text{Monate}] = 2,5 * \text{Aufwand}^{0,35}$
 - Komplex: $TDEV [\text{Monate}] = 2,5 * \text{Aufwand}^{0,32}$
- Achtung: 8 Monate TDEV Minstdauer

- In vorherigen Schritten bestimmt
 - Aufwand für Projekt
 - Entwicklungszeit TDEV
- Personalbedarf P
 - $P = \text{Aufwand [PM]} / \text{TDEV [Monate]}$
- P sagt nichts über Verteilung des Personals aus

COCOMO

Kostentreiberfaktoren

Faktor	von	bis
Zuverlässigkeit	sehr niedrig = 0,75	sehr hoch = 1,4
Komplexität	sehr niedrig = 0,70	sehr hoch = 1,3
Speicherbedarf	kaum = 1,0	hoch = 1,2
Werkzeugverwendung	hoch = 0,90	niedrig = 1,1
Zeitplan	normal = 1,0	schnell = 1,23

Angepasster Wert = Basiswert * (Zuverlässigkeit *
Komplexität * Speicherbedarf *
Werkzeugverwendung * Zeitplan)



COCOMO II - Constructive Cost Model

<http://csse.usc.edu/tools/COCOMOII.php>

Software Size Sizing Method

[SLOC](#) % Design Modified % Code Modified % Integration Required Assessment and Assimilation (0% - 8%) Software Understanding (0% - 50%) Unfamiliarity (0-1)

New

Reused 0 0

Modified

Software Scale Drivers

Precedentedness Architecture / Risk Resolution Process Maturity

Development Flexibility Team Cohesion

Software Cost Drivers

Product **Personnel** **Platform**

Required Software Reliability Analyst Capability Time Constraint

Data Base Size Programmer Capability Storage Constraint

Product Complexity Personnel Continuity Platform Volatility

Developed for Reusability Application Experience

Documentation Match to Lifecycle Needs Platform Experience

Language and Toolset Experience

Project

Use of Software Tools

Multisite Development

Required Development Schedule

Maintenance

Software Labor Rates

Cost per Person-Month (Dollars)

- Basis UML Use Case Diagramme
- Generell bessere Schätzwerte als COCOMO
- Entwickelt von Gustav Karner, 1993
- Unbereinigte Use Case Gewichte (UUCW)
- Unbereinigte Aktor Gewichte (UAW)
- Technischer Komplexitätsfaktor (TCF)
- Umgebungskomplexitätsfaktor (ECF)

Unbereinigte Use Case Gewichte (UUCW)

Use Case Klassifikation	Anzahl von Transaktionen	Gewicht
Einfach	1 bis 3	5
Mittel	4 bis 7	10
Komplex	8 und mehr	15

$$\text{UUCW} = (\text{Gesamtzahl einfacher Use Cases} \times 5) + (\text{Gesamtzahl mittlerer Use Case} \times 10) + (\text{Gesamtzahl komplexer Use Cases} \times 15)$$

Unbereinigte Aktor Gewichte (UAW)

Aktoren Klassifikation	Art des Aktors	Gewicht
Einfach	Externes System muss mit System interagieren mittels wohldefinierter API	1
Mittel	Externes System muss mit System interagieren mittels Standardkommunikationsprotokoll (TCP/IP, REST, ...)	2
Komplex	Menschlicher Aktor verwendet eine GUI zur Interaktion	3

$$\text{UAW} = (\text{Gesamtzahl einfacher Aktoren} \times 1) + (\text{Gesamtzahl mittlerer Aktoren} \times 2) + (\text{Gesamtzahl komplexer Aktoren} \times 3)$$

Technischer Komplexitätsfaktor (TCF)

Faktor Beschreibung		Gewicht	Werte
T1	Verteiltes System	2,0	von 0 (Faktor ist irrelevant) bis 5 (Faktor ist essentiell)
T2	Antwortzeit oder Perfomanz Ziele	1,0	
T3	Endbenutzer effizient	1,0	Gewichtete Summe über Werte ergibt TF
T4	Interne Verarbeitungskomplexität	1,0	
T5	Programmcode Wiederverwendbarkeit	1,0	TCF = 0.6 + (TF/100)
T6	Leichte Installation	0,5	
T7	Leichte Verwendbarkeit	0,5	
T8	Portabilität auf andere Plattformen	2,0	
T9	Systemwartbarkeit	1,0	
T10	Nebenläufige oder parallele Verarbeitung	1,0	
T11	Sicherheitsmerkmale/-anforderungen	1,0	
T12	Zugriff durch Dritte	1,0	
T13	Endnutzerschulungen	1,0	

Umgebungskomplexitätsfaktor (ECF)

Faktor	Beschreibung	Gewicht
E1	Erfahrung mit Entwicklungsprozess vorhanden	1,5
E2	Teilzeitmitarbeiter	-1,0
E3	Begabung der Leitanalysten	0,5
E4	Erfahrung mit Anwendung vorhanden	0,5
E5	Erfahrung in OOP	1,0
E6	Motivation des Teams	1,0
E7	Schwierigkeit der Programmiersprache	-1,0
E8	Stabilität der Anforderungen	2,0

Werte
von 0 (Faktor ist irrelevant)
bis 5 (Faktor ist essentiell)

Gewichtete Summe über
Werte ergibt **EF**

$$\mathbf{ECF = 1.4 + (-0.03 \times EF)}$$

- Einzelne vier Teilfaktoren (UUCW, UAW, TCF, ECF)
- Ergeben die Use Case Points

$$\text{UCP} = (\text{UUCW} + \text{UAW}) \times \text{TCF} \times \text{ECF}$$

- Arbeitsstunden = $\text{UCP} \times F$
 - Umrechnungsfaktor F zwischen üblicherweise 18-28 h/UCP (genauer Wert: Firmengeheimnis)

Part I Critical Estimation Concepts

from Software Estimation: Demystifying the Black Art, by Steve McConnell, p. 36

Projektmanager

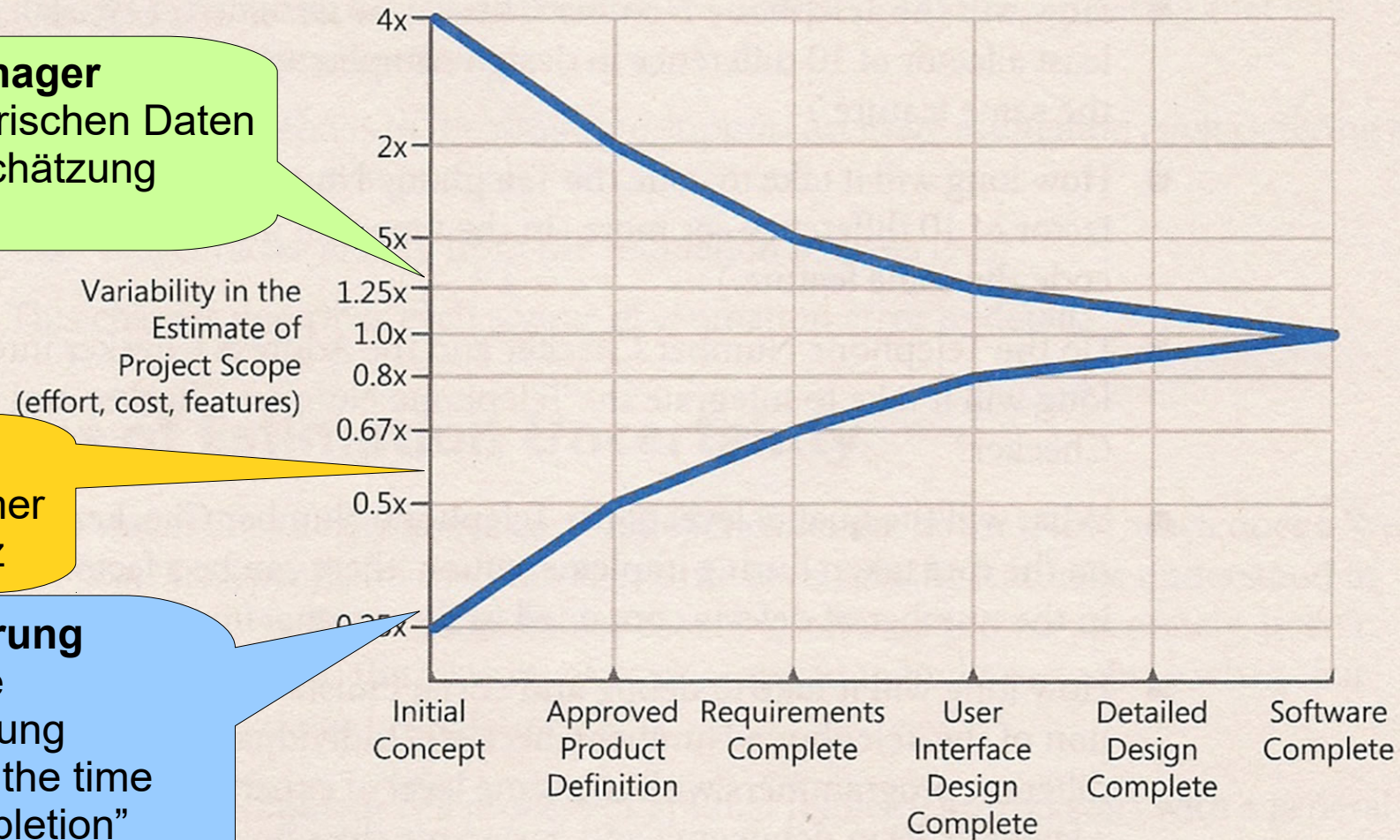
- Schätzen mit historischen Daten
- Versuchen Unterschätzung zu vermeiden

Entwickler

- Unterschätzen immer
- Hofstadters Gesetz

Geschäftsführung

- Parkinson's Squeeze der Entwicklerschätzung
- "Work expands to fill the time available for it's completion" Parkinsons Gesetz



The Cone of Uncertainty based on common project milestones.

In diesem Abschnitt:

- Motivation Softwaremetriken
- Zyklomatische Komplexität
- Werkzeugunterstützung
- Kostenschätzung

Im nächsten Abschnitt:

- Dynamisches Testen

Anhang (weitere Informationen)

Softwarekonstruktion
WS 2016/17



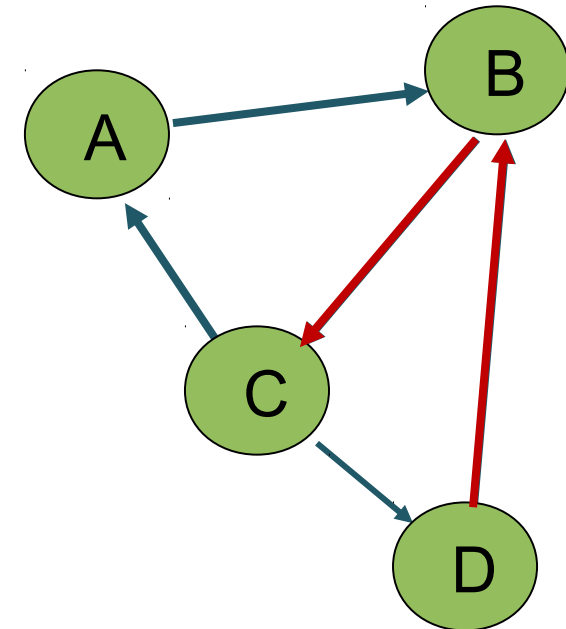
- **Zyklischer Graph:** Auffassung als Vektorraum.
[via Menge seiner Eulerschen Teilgraphen]
→ **Menge unabhängiger Pfade** (erzeugen in Linearkombination alle Pfade durch Graphen).
- **Zyklomatische Zahl:** Anzahl linear unabhängigen Pfade eines (zyklischen) Graphen.
- Sei **$G = (V, E)$** mit
 - $e = |E(G)|$, Anzahl Kanten (edges).
 - $v = |V(G)|$, Anzahl Knoten (vertices).
- Dann gilt **$cn(G) = e - v + 1$ (cyclomatic number).**

- **Alternative (äquivalente) Definition:** Anzahl zu entfernender Kanten stark zusammenhängenden Graphen, um **Spannbaum** zu erhalten

- **Beispiel:**

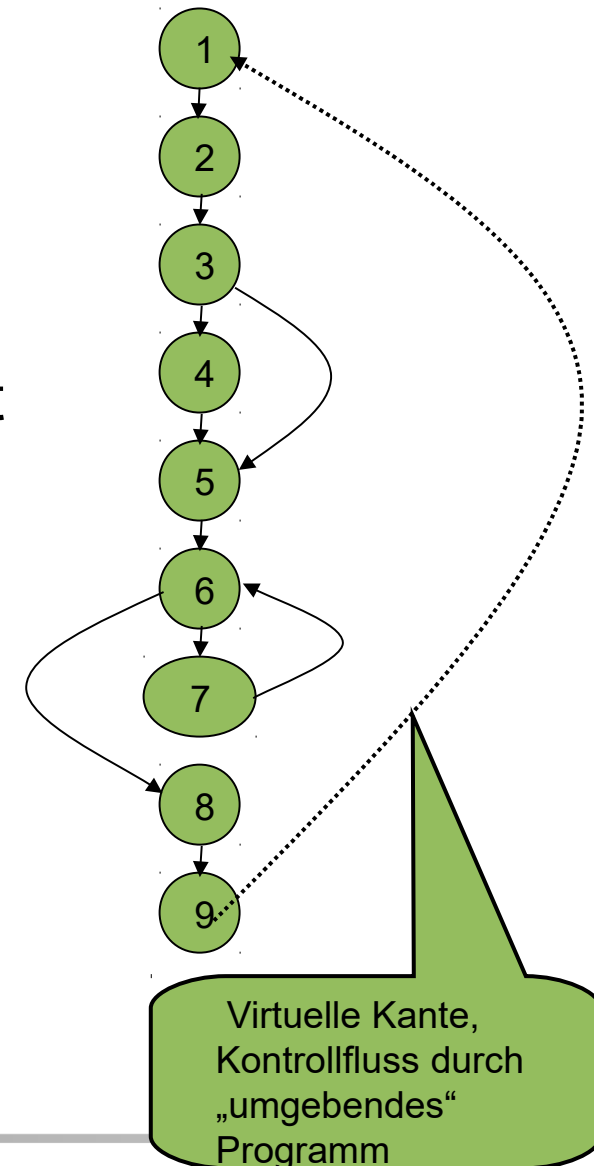
$$cn(G) = 5 - 4 + 1 = 2$$

(NB: **Entfernung beider Kanten B-C und D-B** ergibt einen Spannbaum des Graphen !)



Zyklomatische Komplexität eines Programmes

- Für **Definition dieser Zahl** als Metrik für Programme, Kontrollflussgraphen „**virtuelle**“ **Kante vom Endknoten zurück zum Anfangsknoten** hinzufügen, um zyklischen Graphen zu erhalten. (**Annahme:** Programm hat nur einen Endpunkt.)
- $v(G) = e - v + 2$: **Zyklomatische Komplexität** („McCabe-Metrik“) eines Kontrollflussgraphen G.
- **Beispiel:** $v(G) = 10 - 9 + 2 = 3$.



Bislang Annahme: Programm hat nur ein Endpunkt.

Programm mit mehreren Endpunkten: Für jeden Endpunkt „virtuelle“ Kante zum Startpunkt einfügen. → Zyklischen Graph erhalten.

Zyklomatische Komplexität des Kontrollflussgraphen G eines Programms **mit mehreren Endpunkten:**

$$v(G) = e - n + p + 1$$

- e Zahl der Kanten
- n Zahl der Knoten
- p Zahl der Endpunkte des Programms

[NB: Annahme weiterhin: Nur ein Startpunkt.]

- Größe – Wie umfangreich ist Software?
- Skala: Verhältnisskala.
- Mögliche Maße: **NCSS, Anzahl Zeichen.**

NCSS (Non-commented source statements):

- Zählung der **Codezeilen** ohne Kommentar- und Leerzeilen.
- Genaue **Zählregeln** erforderlich.
- **Programmiersprachenabhängig.**
- **Leicht messbar.**

Softwaremetrik	Beschreibung
Fan-in / Fan-out	Fan-in einer Funktion oder Methode X: Anzahl Funktionen oder Methoden, die X aufrufen. Fan-out : Anzahl Funktionen oder Methoden, die von X aufgerufen werden. Hoher Fan-in : X eng mit Rest des System verbunden. Änderungen an X können weitreichende Folgewirkungen haben. Hoher Fan-out : Gesamte Komplexität von X ist hoch, da Steuerungslogik von X aufgerufene Komponenten koordinieren muss.
Länge der Bezeichner	Maß durchschnittlicher Länge von Bezeichnern (Namen von Variablen, Klassen, Methoden, etc.) im Programm. Je länger sie sind, desto aussagekräftiger sind sie (damit Programm verständlicher).
Tiefe der Verschachtelung	Maß der Tiefe der Verschachtelung von if-Bedingungen im Programm. Tief verschachtelte if-Bedingungen: Schwer zu verstehen und potentiell fehleranfälliger.
Fog-Index	Maß durchschnittlicher Länge von Wörtern und Sätzen im Dokument. Je höher Fog-Index eines Dokuments ist, desto schwerer ist es zu verstehen.

Kürzel	Bezeichnung	Erläuterung
NOV	Number of Variables	Anzahl der Instanzvariablen (member variables) einer Klasse
NOM	Number of Methods	Anzahl der Methoden (Operationen) einer Klasse
NORM	Number of Redefined Methods	Anzahl der in einer Klasse redefinierten Methoden